

Convolutions: The Core Operation

Key Takeaways

1 Local Connectivity

Fully-connected layers on images waste millions of parameters on irrelevant connections. Convolutional layers connect each neuron to only a small local patch, exploiting the fact that nearby pixels are related and distant pixels are not.

2 Convolution = Slide, Multiply, Sum

A small filter (e.g., 3×3) slides across the image. At each position it computes an element-wise product and sums the result into one output value. Different filters detect different patterns (edges, textures, corners).

3 Efficiency Through Sharing and Hierarchy

Parameter sharing means the same filter is reused at every spatial location — a 3×3 filter has only 9 learnable weights regardless of image size. Stacking layers builds a hierarchy: early layers detect edges, deep layers detect objects.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

- Q1.** A $128 \times 128 \times 3$ image is fed into an FC layer with 256 neurons. How many weight parameters does this layer have? Why is this problematic?
- Q2.** An input of size 13×13 is convolved with a 5×5 filter, padding = 2, stride = 2. What is the output size?
- Q3.** Explain why parameter sharing makes sense for image data but would NOT make sense for, say, a fully-connected layer processing tabular features.
- Q4.** Two stacked 3×3 conv layers (stride 1, no padding). What is the effective receptive field on the original input? What about three stacked layers?
- Q5.** What is the main tradeoff of max pooling? In what scenario might pooling hurt performance?

Coding Exercises

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Manual Convolution

Implement a function `conv2d(image, kernel)` from scratch using only NumPy (no deep learning frameworks). Test it on a 7×7 checkerboard image with a 3×3 vertical edge filter. Visualize the input and output side by side using matplotlib. Verify your output matches `scipy.signal.correlate2d`.

Exercise 2: Output Size Calculator

Write a function `output_size(W, H, F, P, S)` that returns the output dimensions after a conv layer. Then write a function `trace_shapes(layer_configs)` that takes a list of (filter_size, padding, stride, num_filters) tuples and prints the shape after each layer, starting from a $32 \times 32 \times 3$ input. Verify with PyTorch.

Exercise 3: Visualize Learned Filters

Load a pretrained `torchvision.models.vgg16`. Extract the weights of the first conv layer (`model.features[0].weight`). Visualize all 64 filters as a grid of 3×3 RGB images. What patterns do you notice? Compare with the filters from `model.features[5]` (layer 2) — are they still interpretable?

Challenge Problem

Build a CNN from Scratch and Compare to FC

1. Build a fully-connected network for MNIST: Flatten $\rightarrow$ Linear(784, 256) $\rightarrow$ ReLU $\rightarrow$ Linear(256, 10). Train for 5 epochs. Record test accuracy and parameter count.
2. Build a CNN for the same task: Conv(1,16,3,pad=1) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2) $\rightarrow$ Conv(16,32,3,pad=1) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2) $\rightarrow$ Flatten $\rightarrow$ Linear(32*7*7, 10). Train for 5 epochs.
3. Compare: (a) test accuracy, (b) total parameters, (c) training time per epoch.
4. **Bonus:** Shift all test images 3 pixels to the right using `torch.roll`. Re-evaluate both models. Which is more robust to translation? Why?

Further Reading

- **Convolution intuition:** LeCun et al., 1998 — *Gradient-Based Learning Applied to Document Recognition*
The original LeNet paper. Section 2 explains convolution beautifully with handwritten digit examples.
- **Receptive fields:** Luo et al., 2016 — *Understanding the Effective Receptive Field in Deep CNNs*
Shows that the effective RF is much smaller than the theoretical RF. Explains why this matters for architecture design.
- **Pooling alternatives:** Springenberg et al., 2015 — *Striving for Simplicity: The All Convolutional Net*
Argues that stride-2 convolutions can replace pooling entirely. A nice perspective on modern CNN design.

Online Resources & Tools

- **CNN Explainer** — Interactive browser visualization of convolution, filters, and feature maps
- **Setosa.io Conv Demo** — Interactive widget to see how filters slide and produce outputs
- **3Blue1Brown: “But what is a convolution?”** — Excellent visual intuition for the operation itself
- **PyTorch Conv2d docs** — API reference with shape formulas and usage examples
- **CS231n Notes (Module 2)** — Stanford’s CNN notes with clear diagrams of padding/stride
- **Distill.pub: Feature Visualization** — Beautiful article on what CNN filters learn at each layer

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Confused about dot products and element-wise operations?** Review Khan Academy’s “Dot product” lesson and NumPy broadcasting rules. The convolution operation is just repeated dot products.
- **Unsure about tensor shapes and dimensions?** Work through PyTorch’s “Tensor Basics” tutorial. Pay attention to how `.shape` changes through layers.
- **Forgot how MLPs work?** Revisit Week 1 Session 2 materials (Perceptron to MLP). CNNs replace the FC layers with conv layers but the overall forward-pass logic is the same.
- **Struggling with the output size formula?** Practice 10 examples by hand with different W , F , P , S values. Use the formula $(W - F + 2P)/S + 1$ and verify with PyTorch’s `nn.Conv2d`.

Answers — Test Your Understanding

- $128 \times 128 \times 3 \times 256 = 12,582,912$ weights. This is problematic because it overfits easily on small datasets, is computationally expensive, and ignores spatial structure entirely.
- Output $= (13 - 5 + 2 \times 2)/2 + 1 = (13 - 5 + 4)/2 + 1 = 12/2 + 1 = 7$. Output size is 7×7 .
- In images, the same pattern (e.g., a vertical edge) can appear anywhere — so one shared filter makes sense everywhere. In tabular data, feature 1 (say, “age”) and feature 5 (say, “income”) have completely different meanings — sharing weights across them would be nonsensical.
- Two 3×3 layers: RF $= 5 \times 5$ (each layer adds 2 pixels of context). Three layers: RF $= 7 \times 7$. Formula: RF $= 1 + L \times (F - 1)$ where L is number of layers and F is filter size.
- Max pooling discards exact spatial location — you know a feature exists in a region but lose precisely where. This hurts in tasks requiring pixel-level precision, like image segmentation or super-resolution, where exact position matters.